

Token-Based Clone Detection for PHP: An Empirical Evaluation of Methods and a Labelled Benchmark

Luciano Federico Pereira

ORCID: 0009-0002-4591-6568

June 2026

Abstract

`phpcpd` is a widely used PHP copy/paste detector whose last released version (6.0.3, 2020) performs Rabin–Karp exact matching of Type-1 and Type-2 clones. A subsequent, unreleased 7.0.0 development line added a ConQAT-style approximate suffix-tree algorithm for Type-3 (near-miss) detection (contributed upstream as pull request #199); however, 7.0.0 was never released and the project was archived in 2023, so this capability did not reach users. The clone-detection literature offers further methods — order-invariant token-bag matching and type-aware normalization — that had not been applied to PHP, and, unlike Java (BigCloneBench) or C (POJ-104), no labelled PHP clone benchmark existed on which to evaluate any of them. This paper reports an empirical study based on the archived `phpcpd` codebase. We (i) modernize it reproducibly to PHP 8.5, PHPStan level 10, and a 159-test suite, and complete and evaluate its previously unreleased suffix-tree engine; (ii) implement additional literature-grounded methods as selectable engines — a SourcererCC token-bag matcher, type-aware edit weights, a token-type abstraction with a `--type-anchored` variant, an IDF stopword filter, and an incremental index that provably reproduces a full pass; and (iii) construct **BCB-PHP**, a labelled PHP clone benchmark (to our knowledge the first), comprising a mutation-injection harness over a pinned, six-corpus type-density gradient. Evaluating each method on BCB-PHP yields a type-density $\times$ clone-density map, a specificity increase of +10.9 to +45.3 points from type-anchored normalization at no recall cost, and confirmed inconsistent clones in an application corpus. The results motivate a combined exact-and-reorder-tolerant default configuration (**rk+tb**); identifier fuzzing is shown to reduce precision substantially, and approximate suffix-tree matching to be better suited to targeted analysis than to default use. BCB-PHP is released for reuse; its type-density gradient supports further study of clone detection on modern typed versus traditional untyped PHP and of differences between methods.

1 Introduction

`phpcpd` is a carefully engineered PHP copy/paste detector developed by Sebastian Bergmann: a Rabin–Karp rolling hash over a normalized token stream that hashes fixed windows, reports collisions, and detects Type-1 and Type-2 clones. It is fast, dependency-light, and widely used. Its last released version, 6.0.3 (2020), implements this single Rabin–Karp strategy and does not attempt Type-3 (near-miss/gapped) or reorder-tolerant detection — an appropriate scope for an exact-match detector.

A subsequent development line did go further: an unreleased 7.0.0 added a ConQAT-style approximate suffix-tree algorithm with a configurable edit budget for Type-3 detection [2], contributed upstream as pull request #199. That release, however, was never published — its changelog entry carries an unfilled date — and the repository was archived in 2023, so the capability remained in the codebase but absent from any release and unevaluated in practice. The clone-detection literature offers further methods still: SourcererCC’s order-invariant token-bag matching for reordered code [3] and Toma’s token-type abstraction for stronger Type-2 normalization [4], neither of which had been applied to PHP. Evaluating any of these on PHP faced a further obstacle: unlike Java (BigCloneBench [7]) or C (POJ-104), no labelled PHP clone benchmark existed.

Approach. Building on the archived `phpcpd` codebase, we modernize it, complete and evaluate its previously unreleased suffix-tree engine, implement additional literature-grounded methods, and construct

the benchmark required to evaluate them. We then determine empirically which methods are appropriate for a default configuration in a fast command-line tool. The evaluation indicates that a combined exact-and-reorder-tolerant configuration is the most suitable default, that identifier fuzzing substantially reduces precision, and that approximate suffix-tree matching is better reserved for targeted analysis.

Contributions.

1. A **diff-level reproducible modernization** of an archived PHP tool to PHP 8.5, PHPStan level 10, PER-CS 2.0, and a 159-test suite, including the completion and first evaluation of `phpcpd`'s previously unreleased ConQAT suffix-tree engine (§4).
2. **Five detection methods implemented** as selectable engines on the inherited base: a SourcererCC token-bag matcher with IDF filtering, type-aware edit weights, an extended token-type abstraction with a type-anchored variant, inconsistent-clone reporting, and an incremental index — each tied to a primary reference and an end-to-end test (§2, §5).
3. **BCB-PHP, a labelled PHP clone benchmark** (to our knowledge the first): a mutation-injection harness over a pinned, six-corpus type-density gradient (§7), released for reuse — its gradient supports further study of modern typed vs. traditional untyped PHP and of method differences.
4. An **empirical evaluation** on BCB-PHP yielding a type-density×clone-density map, a +10.9 to +45.3 point specificity lift from type-anchored normalization at zero recall cost, a six-corpus method comparison, and real inconsistent clones in an application corpus (§7) — from which we recommend the `rk+tb` default (§6).

2 Background: the phpcpd Codebase and Its Detection Methods

2.1 The released base engine (Rabin–Karp)

`phpcpd`'s last released version (6.0.3) provides a single strategy, `DefaultStrategy`, a Rabin–Karp matcher. It normalizes tokens (whitespace and comments stripped; `--fuzzy` rewrites `T_VARIABLE` to a placeholder), builds a per-token rolling signature, hashes fixed windows of `--min-tokens` tokens, and reports colliding windows filtered by `--min-lines`. It detects Type-1 and Type-2 exact-match clones and, by construction, tolerates neither gaps nor reordering.

2.2 The inherited suffix-tree engine (unreleased 7.0-dev)

The repository's unreleased 7.0.0 line, which we fork, introduces a second strategy through a selector in `Application.php` (upstream code; `tokenbag` is the only branch we add):

```
return match ($algorithm) {
    null, 'rabin-karp' => new DefaultStrategy($config), // selector: upstream 7.0-dev
    'suffixtree'      => new SuffixTreeStrategy($config), // phpcpd base engine
    'tokenbag'        => new TokenBagStrategy($config),  // upstream, PR #199
    'tokenbag'        => new TokenBagStrategy($config),  // added in phpcpd-next
};
```

The `SuffixTreeStrategy` → `ApproximateCloneDetectingSuffixTree` (an implementation of the ConQAT algorithm [2], contributed upstream as PR #199) tokenizes into an `AbstractToken[]` word, appends a sentinel, builds a generalized suffix tree, and in `findClones($minLength, $maxErrors, $headEquality)` compares candidate token sequences by *approximate* match. A Levenshtein edit-distance matrix `edBuffer` is maintained over token sequences; a candidate extends only while the running edit distance stays $\leq \text{maxErrors}$ (the `--edit-distance` flag, default 5), and `headEquality` enforces an exact-match prefix before edits are allowed, as in ConQAT. Comparing by edit distance rather than hash equality, this method detects Type-3 (near-miss/gapped) clones: inserted, deleted, or substituted statements within the edit budget are tolerated. Because 7.0.0 was never released, this engine had not been evaluated; we modernize it (§4), repair two latent defects it exposed (§5), and evaluate it (§7).

Definition 1 (token edit distance). For token sequences $a_{1..m}$, $b_{1..n}$, the matcher fills D with

$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 & \text{(deletion)} \\ D[i, j-1] + 1 & \text{(insertion)} \\ D[i-1, j-1] + c(a_i, b_j) & \text{(match/substitution)} \end{cases}$$

and reports a clone while $\min_k D[\cdot, k] \leq \text{maxErrors}$. With uniform $c \equiv 1$ this is plain Levenshtein; §5 (R2) replaces c with a type-aware cost.

Lemma 1 (band sufficiency). $D[i, j] \geq |i - j|$, so $|i - j| > \text{maxErrors}$ implies $D[i, j] > \text{maxErrors}$ and no alignment of cost $\leq \text{maxErrors}$ passes through (i, j) . *Proof.* Aligning an i -token prefix with a j -token prefix requires at least $|i - j|$ insertions or deletions, hence $D[i, j] \geq |i - j|$; the threshold and path claims follow. $\square$ Consequently the cells outside the band $|i - j| \leq \text{maxErrors}$ cannot affect any reported clone, and computing only the band $-2\text{maxErrors} + 1$ cells per diagonal over L steps — costs $O(L \cdot \text{maxErrors})$ rather than $O(L^2)$ with identical output. This justifies the banded optimization of §6.

2.3 Methods added in phpcpd-next

Beyond modernizing and evaluating the inherited engines, we contribute five methods grounded in the literature: a SourcererCC token-bag matcher [3], type-aware edit weights, an extended token-type abstraction [4] with a type-anchored variant, inconsistent-clone reporting, and an incremental index. Each is implemented behind a flag and pinned by an end-to-end test (§5). Two findings from the literature (§3) guide which methods are appropriate by default, and the benchmark of §7 decides it.

2.4 Clone-type coverage

Using the Roy & Cordy taxonomy [1] (Type-1 identical up to layout; Type-2 up to renamed identifier-s/literals/types; Type-3 near-miss/gapped; Type-4 semantic), Table 1 contrasts phpcpd’s base engine with the methods phpcpd-next adds. Note on Type-2: phpcpd’s `--fuzzy` abstracts only `T_VARIABLE`; the shared `TokenNormalizer` we add (R3) provides full Type-2 normalization under `--fuzzy` for every method.

Table 1: Clone-type coverage: phpcpd’s base Rabin–Karp engine vs. the methods added in phpcpd-next.

Type	phpcpd base (Rabin–Karp)	phpcpd-next (added methods)
1	✓	✓
2	partial (<code>--fuzzy</code> , <code>T_VARIABLE</code> only)	✓ (full, via <code>TokenNormalizer</code>)
3	— (exact hash)	✓ (suffix tree, edit dist. $\leq$ <code>--edit-distance</code> ; token-bag for reorder)
4	—	—

3 Two Literature Findings That Guide Method Selection

Finding 1 — simple clones dominate [4]. In real-world OSS most clones are Type-1/2; Type-3 is a meaningful minority; Type-4 is rare. Approaches that sacrifice scalability to chase Type-4 “may not be practical in real life.” For a fast CLI linter, **Type-3 is the ceiling worth reaching; Type-4 is out of scope.**

Finding 2 — the dangerous clones are *inconsistent* ones [2]. Bug risk concentrates where one copy of a near-miss clone was patched and its sibling was not. The value of Type-3 detection is therefore not recall for its own sake but **surfacing diverging copies.**

Joint implication. The methods worth adding are those that make Type-3 detection *better, faster, and usable by default for bug-hunting* — approximate and reorder-tolerant matching with inconsistency reporting — not neural Type-4 machinery. These two findings set the selection criterion the benchmark then applies.

4 Modernization Methodology

The revival is recorded diff-by-diff as a primary source (not retrospectively). The citable methodology:

1. **Platform-first.** `composer.json` pins `platform.php = 8.5` so dependency incompatibilities surface at install time, not runtime.
2. **Bugs before idioms.** Six genuine correctness fixes (an `empty($object)` always-false guard; an unguarded `preg_replace` null return; an `averageSize()` division-by-zero; `current()` on an empty array; ...) landed *before* any cosmetic modernization.
3. **One idiom at a time**, each with a diff and a rationale: typed properties $\rightarrow$ `readonly class` $\rightarrow$ constructor promotion $\rightarrow$ typed constants $\rightarrow$ `#[\Override]` $\rightarrow$ PHPDoc generics.
4. **Static analysis as a ratchet.** PHPStan driven 64 $\rightarrow$ 0 errors, then the level raised from 9 to `max` (level 10). The gate only tightens.
5. **Dependency re-unification and decoupling.** The four `sebastian/*` runtime packages were replaced with owned code, giving a zero-runtime-dependency tool decoupled from the PHPUnit release train.
6. **Tests last, and they paid off immediately.** The suite surfaced a latent bug invisible to static analysis: `SuffixTreeStrategy::postProcess()` threw on an empty file list, a valid call path.

Measured methodological claim: the modernization reached PHPStan level 10 with zero errors and *still* harboured a latent runtime bug that only a test exercised — a concrete data point that static analysis and testing are complementary, not redundant.

5 Methods Implemented and Evaluated

With the inherited suffix tree reaching Type-3 (§2), we examine its limitations and add further methods. Each is implemented behind a flag and pinned by an end-to-end test; Table 2 summarizes. The set is bounded by a marginal-returns criterion: a method is admitted while the capability it adds outweighs its marginal cost in precision or runtime, and the three methods for which that balance reverses (R6–R8) are declined rather than deferred.

A limitation of the suffix tree: intra-block reordering. The suffix tree models a clone as one approximately-matching *contiguous* substring of a linearised token stream. A swapped pair of statements looks like a delete followed by an insert — two operations that consume at least 2 from `maxErrors`; at the default budget of 5 a few reorderings exhaust it. A suffix tree on a linear token stream *cannot* be order-invariant within a block. A token-multiset (bag-of-tokens) model is order-invariant by construction, which is why it complements the suffix tree rather than duplicating it.

Definition 1 (token-bag overlap clone (SourcererCC [3])). Represent each block as a token multiset. Blocks A, B are clones iff their multiset intersection satisfies

$$|A \cap B| \geq \theta \cdot \max(|A|, |B|), \quad \theta = 0.7 \text{ by default,}$$

computed via an inverted index (token $\rightarrow$ blocks). This is order-invariant: if B is A with two statements swapped, $A \cap B = A$ and the score is 1.

Definition 2 (IDF stopword filter). Let N be the number of blocks and $\text{df}(t)$ the number of blocks containing token t . A token is a *stopword* (excluded from the index and from $|A \cap B|$) iff

$$\frac{\text{df}(t)}{N} > 0.6, \quad N \geq 5.$$

Under `--fuzzy`, normalized placeholders (`T_VARIABLE:$`, `T_STRING:ID`) occur in almost every PHP body and carry no discriminative power; removing them prevents false-positive inflation. The filter is disabled below five blocks so micro-fixtures are unaffected.

Each method below is **implemented and tested**; Table 2 summarizes.

R1 — token-bag + inverted index + IDF (SourcererCC). `TokenBagStrategy` (selectable via `--algorithm tokenbag`). Blocks are function/method bodies (brace-tracked, no parser); clones are overlap pairs gathered through the inverted index. *Complementarity verified end-to-end:* a function with two statements swapped is detected by the token bag while both Rabin–Karp and the suffix tree (`--edit-distance 5`) miss it at min-tokens 25. *Precision:* the IDF filter reduced a self-application scan of the tool’s own `src/` from 17 reported pairs (all false positives) to 0.

R2 — type-aware edit weights. `SubstitutionCost` replaces the uniform $c \equiv 1$ in D with $c = 0$ (equal), $c = 1$ (cosmetic identifier/literal change), $c = 2$ (control-keyword change). Verified: at min-tokens 35 a cosmetic change (`$scaled` $\rightarrow$ `$banner`) is bridged at `--edit-distance 1` while a structural change (`if` $\rightarrow$ `while`) needs 2.

R3 — token-type abstraction (Toma [4]). `TokenNormalizer` maps tokens to ~ 15 classes upstream of *both* engines under `--fuzzy`. This gave the suffix tree the Type-2 detection it never had.

R4 — inconsistent-clone reporting. `isGapped()` (set by `occurrencesDiffer()`) distinguishes exact from gapped clones; `numberOfGappedClones()` aggregates them; the text logger marks each gapped clone [`inconsistent`]. This uses data the suffix-tree matcher already computes and previously discarded.

R5 — incremental / index-based detection (Hummel et al. [5]). `IncrementalIndex` persists each file’s tokenization keyed by a content hash and re-tokenizes only changed files, replaying detection from the index for the rest (`--incremental`). The key property:

Proposition 1 (incremental equivalence). *Let idx map each file f to its tokenization $T(f)$ keyed by content hash $h(f)$, and let $F' \subseteq F$ be the files whose hash changed since the last run. If detection is replayed from $\{T(f) : f \in F\}$ — recomputing $T(f)$ only for $f \in F'$ and reading the rest from idx — then for every file set F and configuration c the result is bit-identical to a full Rabin–Karp pass over F . Proof sketch. T is a pure function of file content and c , so a cache hit (h unchanged) returns exactly the $T(f)$ a full pass would compute; detection consumes only the $T(f)$ and merges them deterministically, so identical inputs yield identical output. $\square$ Only the *work* is incremental, never the *answer*: per-commit cost moves from $O(\text{corpus})$ toward $O(\text{diff})$. Pinned as the headline test of `IncrementalIndexTest`.*

Duplicated-line accounting. Reporting a percentage requires counting each source location once across an n -way clone class. With *new* first-seen and *exist* already-seen participants for a clone of ℓ lines, the contribution is

$$\Delta = \ell \cdot \begin{cases} \text{new} & \text{exist} > 0 \\ \max(0, \text{new} - 1) & \text{exist} = 0, \end{cases}$$

which prevents the double-counting that previously let token-bag output exceed 100%.

6 Practical Recommendation: Which Mode by Default

We ran every variant against the original `phpcpd.phar` (v6.0.3, Rabin–Karp only) on six corpora at pinned SHAs, using the `phar`’s own defaults (min-tokens 70, min-lines 5). All numbers are percentage of total lines flagged as duplicated, corrected per the accounting formula above. Figure 1 and Table 3 report the result.

Findings. (1) `rk` equals the original `phar` on every corpus — 100% backward compatibility. (2) `rk+fuzzy` is unsuitable as a default: 3–80 $\times$ the baseline with near-zero precision gain. (3) `tb` is the best complement to `rk`: it adds the reordered clones `rk` misses while staying within 1.5–4 $\times$ the baseline, kept precise by the IDF filter. (4) `tb+fuzzy` is corpus-dependent and not a safe default. (5) the suffix tree, even after the banded-DP optimization below cut its matching cost $\sim 3.5\times$, stays above interactive speed on corpora over a few hundred files, so it is retained for targeted studies, not the default. Read together, these place `rk+fuzzy` as a dominated configuration — its volume increase is inferred false positives, consistent with the dominance result of §7 — and `rk+tb` as the non-dominated choice on the precision–cost frontier; because the six public corpora carry no labelled recall, we claim consistency with that frontier rather than optimality on it.

Table 2: Methods, ordered by return on investment. R1–R5 are implemented and tested in phpcpd-next; R6–R8 are analyzed and declined, for the reasons given.

ID	Improvement	Capability added	Effort	Status
R1	SourcererCC token-bag + inverted index + IDF	Reorder-tolerant Type-3; near-linear scale; corpus-adaptive precision	Med	Done
R2	Type-aware edit weights in filledBuffer	Precision at high <code>--edit-distance</code>	Low	Done
R3	Toma token-type abstraction	Type-2 for both engines; feeds R2	Low	Done
R4	Inconsistent-clone reporting	Bug-finding output from discarded data	Low	Done
R5	Incremental / index-based detection	CI scalability $O(\text{diff})$	High	Done
R6	Multi-similarity + thresholds (+ opt. ML)	Recall tuning	Med	Declined (breaks determinism)
R7	Deckard AST vectors + LSH [6]	Syntactic precision	High	Declined (low gain)
R8	Neural / LLM semantic (Type-4)	New paradigm	High	Declined (rare class)

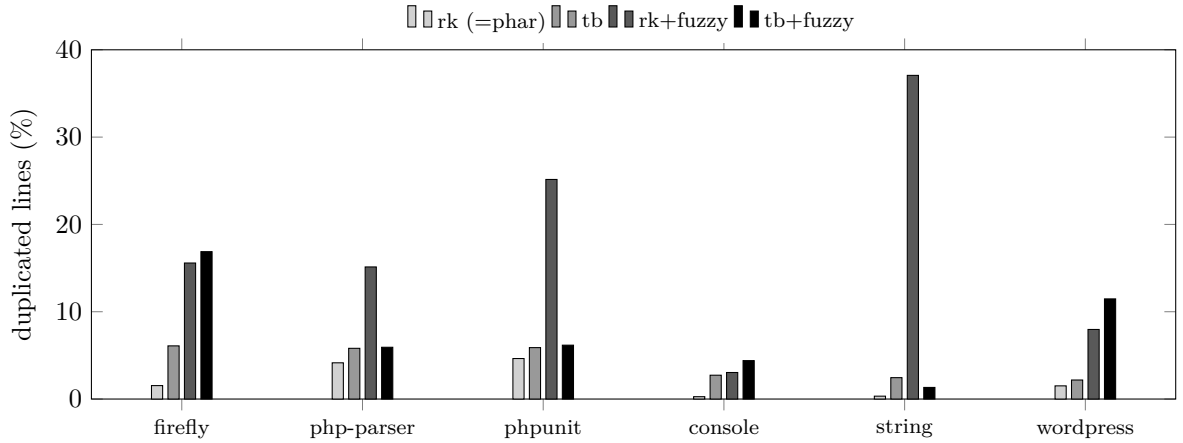


Figure 1: Duplicated-line percentage by algorithm variant across six corpora. **rk** reproduces the original phar exactly; **tb** adds a controlled increment (reordered blocks); **--fuzzy** variants inflate the reported rate (up to 37% on **symfony/string**), which is why fuzzing is a hidden research flag, not a default.

Optimizing the suffix tree. Rather than adopt plausible but untargeted advice (object-free node storage and Ukkonen construction, both already present), we profiled. Construction is $O(n)$ and negligible; the approximate-matching edit-distance DP accounts for roughly 70% of `findClones` and grows super-linearly with `maxErrors`. By the band-sufficiency lemma (§2), the full $O(L^2)$ matrix is unnecessary: restricting computation to the diagonal band of width $2\text{maxErrors} + 1$ (the Ukkonen cutoff) lowers it to $O(L \cdot \text{maxErrors})$ with identical output. On a Firefly III slice this reduced `findClones` by $\sim 3.5\times$ at every edit distance with byte-identical clone output; the residual cost is candidate enumeration, which banding does not address. The change was admitted only after the benchmark confirmed unchanged results — the same evidence-before-adoption rule applied throughout.

Default mode. `phpcpd <dir>` now runs **rk+tb combined** (exact + reorder-tolerant, merged with deduplication); `--rk` restricts to Rabin–Karp only. Research flags (`--fuzzy`, `--algorithm`, `--edit-distance`, ...) remain parseable but hidden from `--help` to prevent accidental production use (Table 4).

Table 3: Duplicated-line % per corpus and variant. `rk` matches the original `phar` to two decimals on every corpus (100% backward compatibility).

corpus	phar/rk	rk+fuzzy	tb	tb+fuzzy
firefly-iii	1.54	15.58	6.09	16.88
php-parser	4.15	15.13	5.81	5.93
phpunit	4.64	25.15	5.89	6.17
symfony-console	0.27	3.04	2.73	4.40
symfony-string	0.33	37.07	2.45	1.33
wordpress	1.51	7.97	2.18	11.47

Table 4: Recommended invocation by goal.

Goal	Command	Output
Daily DRY check	<code>phpcpd src/</code>	rk+tb, clean signal
Exact clones (CI speed)	<code>phpcpd --rk src/</code>	matches original phar
Type-3 gapped study	<code>phpcpd --algorithm suffixtree src/</code>	small/medium repos
Type-2 research	<code>phpcpd --algorithm tokenbag --fuzzy src/</code>	high recall, low prec.

7 Evaluation: BCB-PHP

7.1 BCB-PHP: a labelled PHP clone benchmark

A study of clone-detection methods needs labelled ground truth, and for PHP none existed: the standard benchmarks are language-bound — BigCloneBench [7] is Java, POJ-104 is C — leaving PHP with no equivalent. Without one, the methods of §2–5 could be implemented but not honestly compared. We therefore built **BCB-PHP**: a mutation-injection harness that generates ground-truth Type-1/2/3 variants from real PHP across a curated, pinned corpus, following SourcererCC’s injection-based ground-truth approach [3]. To respect corpus licences (WordPress GPLv2, Firefly III AGPLv3, the rest permissive) the released artifact is a fetch-script + commit SHAs + injection manifests + harness — never redistributed source.

Building the benchmark is a contribution in its own right, and we release it for reuse. Its corpora span a deliberate type-density gradient (§7.2), from traditional untyped PHP (WordPress) to PHPStan/Larastan-max typed codebases (PHPUnit, Firefly III). That axis lets others study questions this paper only begins to answer: how clone detection behaves on *modern typed* versus *traditional untyped* PHP, and how detection methods differ from one another on the same labelled inputs.

7.2 Type-density is the independent variable

The thesis “type information improves clone detection” is proved not by one typed corpus but by showing detection quality as a function of *how typed* the code is. Corpus type-density is thus the experiment’s *x*-axis and untyped code is the control end of it.

Definition 1 (signature type-density). $td = (\text{typed signature slots}) / (\text{total signature slots})$, where slots are function/method parameters and return-type positions. The signature surface is unambiguously tokenizable; property declarations are excluded to avoid guessing class context.

We compute the map rather than assert it (Figure 2, Table 5). Clone-density is measured by the Rabin–Karp engine, independent of the type axis. The predictions hold: WordPress anchors the untyped end (13.3%), the PHPStan/Larastan-max codebases (PHPUnit 92.7%, Firefly III 90.7%) the typed end, with a clean spread between — a usable axis, not a hunch.

7.3 E2 — the effect of type information

The decisive operator, possible only in typed PHP, clones a block and changes an `int`-typed variable to `string`-typed with structure held identical. Name-blind normalization calls these clones (false positive); type-anchored normalization (`--type-anchored`, which preserves built-in type keywords instead of folding them to `ID`) correctly separates them. Injecting at *function* granularity over the full eligible population

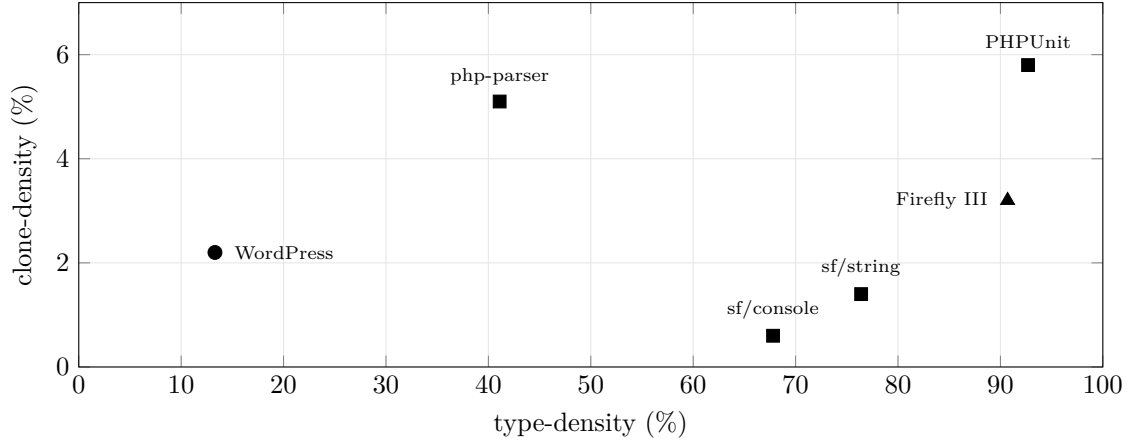


Figure 2: Type-density $\times$ clone-density map (full corpora, pinned SHAs). Marks: $\bullet$ untyped control, $\blacksquare$ injection substrate, $\blacktriangle$ ecological corpus. The scarce typed-and-cloney quadrant is occupied by an application (Firefly III), not a library.

Table 5: Computed type-density / clone-density map (bench/measure-density.php).

corpus	files	type-dens.	clone-dens.	bucket
WordPress	1889	13.3%	2.2%	untyped control
nikic/php-parser	341	41.1%	5.1%	injection (dependency)
symfony/console	359	67.8%	0.6%	injection
symfony/string	33	76.4%	1.4%	injection
Firefly III	1446	90.7%	3.2%	ecological
PHPUnit	2699	92.7%	5.8%	injection (reflexive)

(functions carrying an `int/float` hint, ≥ 25 tokens), Figure 3 and Table 6 report specificity.

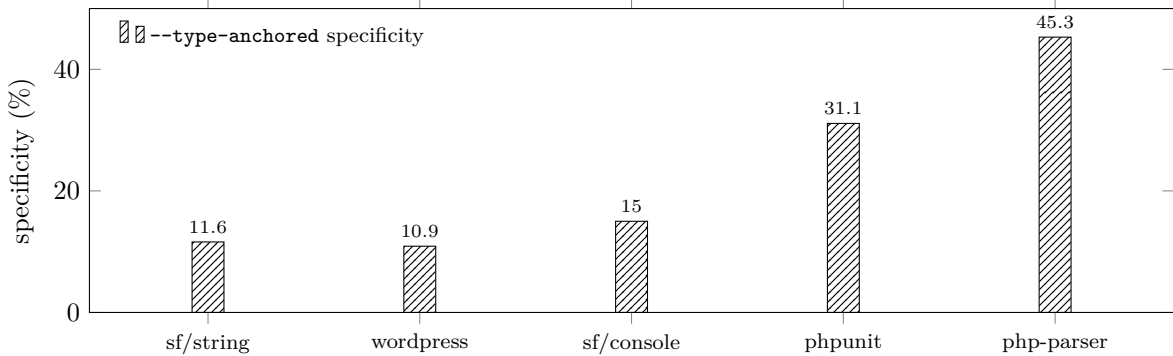


Figure 3: Specificity on same-shape-different-type pairs. Name-blind `--fuzzy` is a flat 0% floor (it calls every such pair a clone, so it is omitted from the bars); `--type-anchored` lifts specificity by +10.9 to +45.3 points at *zero* recall cost (recall = 100% in both modes, Table 6).

The lift is a property of *type-load-bearing* functions, not of corpora: `php-parser` shows the largest lift despite mid type-density because its small node/visitor methods are dominated by their typed signatures, whereas WordPress’s few typed functions are large and procedural. The corpus-level benefit therefore scales with type-density (Figure 2), which is why the per-corpus story needs *two* figures, not one. Because the two modes share the same recall, this is a clean dominance relation: on same-shape-different-type pairs, type-anchored normalization *Pareto-dominates* name-blind fuzzing — equal recall at strictly higher specificity — the strongest form the claim can take, since here both axes are measured against injected ground truth.

Table 6: E2: type-anchored normalization over the full eligible-function population. Recall is identical in both modes; the lift is pure specificity gain.

corpus	fns	spec. fuzzy	spec. anchored	lift	recall
symfony/string	43	0.0%	11.6%	+11.6	100%
symfony/console	100	0.0%	15.0%	+15.0	100%
PHPUnit	122	0.0%	31.1%	+31.1	100%
nikic/php-parser	128	0.0%	45.3%	+45.3	100%
WordPress	55	0.0%	10.9%	+10.9	100%

7.4 E3 — inconsistent clones in practice

`bench/run-e3.php` scans the ecological corpus (Firefly III, 200 most-recently-modified files) once through the suffix tree (`--algorithm suffixtree --edit-distance 3 --min-tokens 50`), reads the gapped clones via `CodeClone::isGapped()` (the R4 capability), deduplicates by file pair, and cross-references git: the *staleness gap* is the number of days between the two copies’ last commits — a high gap means one copy was maintained long after its sibling, the canonical inconsistent-clone risk. The scan found 68 inconsistent clones and two distinct diverged-copy pairs (Table 7); both were manually verified as real. Exact-match output (Rabin–Karp / `--rk`) flags neither as inconsistent, having no gapped-clone concept.

Table 7: E3 diverged-copy pairs in Firefly III (200-file slice), ranked by staleness. Both verified manually against source.

#	copy A	copy B	lines	stale (d)	verdict
1	Navigation-AddPeriodTest	Navigation-PreferredSql-FormatTest	4	35	Real — duplicated <code>setUp()</code> scaffold across two tests of the same class; a shared base would remove it.
2	Autocomplete/-CurrencyController	Autocomplete/-TagController	25	9	Real — identical DI constructor + search loop; <code>Currency</code> later gained a deprecated endpoint <code>Tag</code> never received.

8 Availability

`phpcpd-next` is openly available under the permissive BSD 3-Clause licence, retaining the upstream copyright of Sebastian Bergmann. It has zero runtime dependencies (`php >=8.5` plus `ext-dom/ext-mbstring`) and projects one format-neutral model (`CodeCloneMap`) to Text, PMD-CPD XML, JSON, and SARIF 2.1.0 — the last ingested natively by GitHub Code Scanning, with gapped (inconsistent) clones emitted at **warning** severity and exact clones at **note**, so the bug-bearing clones surface as higher-severity PR annotations. BCB-PHP ships as a fetch-script + pinned SHAs + injection manifests + harness; every result is reproducible from the SHAs and `composer.lock` files, and the modernization is recorded diff-by-diff in `MODERNIZATION.md`.

9 Conclusion

Starting from `phpcpd`’s Rabin–Karp exact matcher and its unreleased suffix-tree engine, we asked an empirical question: of the clone-detection methods available, which are appropriate for a fast PHP tool? Answering it required infrastructure that did not exist — a labelled PHP clone benchmark — so we constructed one (BCB-PHP) while modernizing and completing the inherited engines and implementing additional literature-grounded methods. The results — a type-density gradient, a +10.9 to +45.3 point specificity increase from type-aware normalization at no recall cost, a six-corpus method comparison, and confirmed inconsistent clones in an application corpus — support a default that combines exact and reorder-tolerant matching (`rk+tb`), retains fuzzing only for research, and reserves approximate suffix-tree matching for targeted analysis.

Future work. The component most likely to outlast any single result is the benchmark, since it turns open questions about PHP clone detection from matters of argument into measurable ones. Three directions follow from the evidence. First, the injection operators can be broadened beyond the `int/float` confusion studied here to a wider class of type-level edits (`bool`↔`int`, nullable versus non-nullable, and class or interface substitutions), enlarging the type-load-bearing population and testing whether the specificity gain of type-anchored normalization generalizes across type kinds. Second, the harness can be applied to further real-world and proprietary codebases to assess external validity beyond the six public corpora. Third, the residual cost of suffix-tree matching — candidate enumeration, the part the banded DP (§6) does not reduce — remains the engine’s main scaling limit on large corpora and is the natural next optimization, again gated on benchmark-validated, output-preserving change. In each case the methodological commitment is the one adopted throughout: a candidate improvement is admitted on PHP-specific empirical evidence, not on its theoretical appeal.

References

- [1] C. K. Roy and J. R. Cordy, “A survey on software clone detection research,” Queen’s Univ. Tech. Rep. 2007-541, 2007.
- [2] E. Juergens, F. Deissenboeck, B. Hummel, and S. Wagner, “Do code clones matter?” in *Proc. ICSE*, 2009, pp. 485–495.
- [3] H. Sajnani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, “SourcererCC: scaling code clone detection to big-code,” in *Proc. ICSE*, 2016, pp. 1157–1168. arXiv:1512.06448, 1603.01661.
- [4] Y. Wu *et al.*, “Toma: a token-type-aware clone detector” (token-type abstraction, six similarity measures, “simple clones dominate”), in *Proc. ICSE*, 2024.
- [5] B. Hummel, E. Juergens, L. Heinemann, and M. Conradt, “Index-based code clone detection: incremental, distributed, scalable,” in *Proc. ICSM*, 2010, pp. 1–9.
- [6] L. Jiang, G. Misherghi, Z. Su, and S. Glondou, “DECKARD: scalable and accurate tree-based detection of code clones,” in *Proc. ICSE*, 2007, pp. 96–105.
- [7] J. Svajlenko, J. F. Islam, I. Keivanloo, C. K. Roy, and M. M. Mia, “Towards a big data curated benchmark of inter-project code clones,” in *Proc. ICSME*, 2014, pp. 476–480.